ToddySoft Connect UMAS Driver - Ignition 8.3

Ignition 8.3 module for connecting to Schneider Electric **Modicon** controllers — M340, M580, Premium and Quantum — over UMAS, the native protocol Control Expert itself uses. Tags are addressed by the **symbol names from the PLC program**, not by register numbers: the driver downloads the controller's data dictionary and browses it.

The device configuration uses a custom configuration form: settings that do not apply to what you have selected stay out of your way, and the fields that carry a security consequence say so where you read them.

> **UMAS and Modbus share port 502.** They are multiplexed onto the same connection through different function codes, so a controller you can already reach with the Modbus module needs no extra port opening for this one. What they give you differs completely: Modbus offers numbered registers, UMAS offers the program's own symbols and types.

Installation

> **This module is for Ignition 8.3 only.** It is built against the Ignition 8.3 SDK and will not run on any other major version. For Ignition 8.1, install the separate `…-8.1.modl` build instead — the two are not interchangeable, and each gateway needs the build that matches it.
>
> **Ignition 8.0 and earlier are not supported and cannot be.** The module is built on the device-driver API that Ignition introduced with 8.1, so on 8.0 the gateway fails to load it with `NoClassDefFoundError: … opcua.server.api.AbstractDeviceModuleHook`.
>
> Both builds carry the same name in the gateway's module list, so tell them apart by the file you installed or by the module description, which names the Ignition version.
>
> **Ignition's OPC UA module must be installed and running.** This is an OPC UA device driver: it registers with Ignition's own OPC UA server, and every tag it exposes is served through it. With that module absent the gateway cannot load this one, and the log fills with `ClassNotFoundException` / `NoClassDefFoundError` naming `com.inductiveautomation.opcua…` classes. The OPC UA module ships with Ignition and is installed by default, so in practice this bites only on a gateway where it has been removed or disabled. This module declares the dependency as **required** — a mechanism 8.3 has and 8.1 lacks — so Ignition does not attempt to load it at all, and reports the reason instead of failing with a stack trace. In **Status > Diagnostics > Logs** you will see:
>
> `Module "com.toddysoft.connect.…" depends on "com.inductiveautomation.opcua", which is either not installed or inactive on this gateway.`
>
> This is the same handling Ignition's own drivers get in the same situation.

1. Open the Ignition Gateway web interface and navigate to **Config > System > Modules**.
2. Click **Install or Upgrade a Module...**.
3. Select the `ToddySoft-Connect-UMAS-Driver-8.3.modl` file and upload it.
4. Accept the license / certificate prompt to continue the installation.
5. Wait for the Gateway to finish and verify the module appears as **Running** in the modules list.

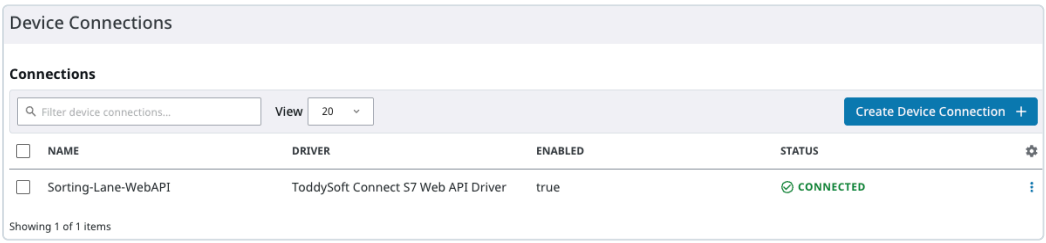
Before you start: the controller side

- **The data dictionary must be enabled in the PLC project.** Browsing reads the symbol table the controller carries in its data dictionary, and a project built without one has no symbols to hand out — the device connects, and the browse is simply empty. Enable it in EcoStruxure Control Expert under **Tools → Project Settings → General → PLC embedded data → Data dictionary**, then rebuild the project and download it. The setting takes effect only with a download, so enabling it alone changes nothing on a controller that is already running.
- **The PLC program must be running.** The data dictionary is downloaded from the controller, so a stopped or empty program leaves nothing to browse and no symbol names to address.
- **UMAS must not be blocked.** Schneider controllers permit it by default, but some site security policies disable it — in which case the TCP connection succeeds and the protocol exchange does not.
- **Port 502 reachable** from the gateway host.

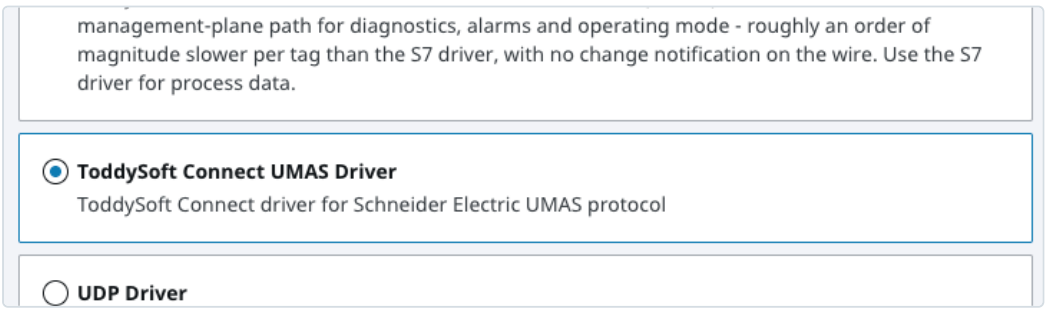
There are no credentials to configure: UMAS as this driver speaks it is unauthenticated, which is why this module has no username or password fields at all.

Creating a Device Connection

1. Go to **Config > OPC UA > Device Connections**.

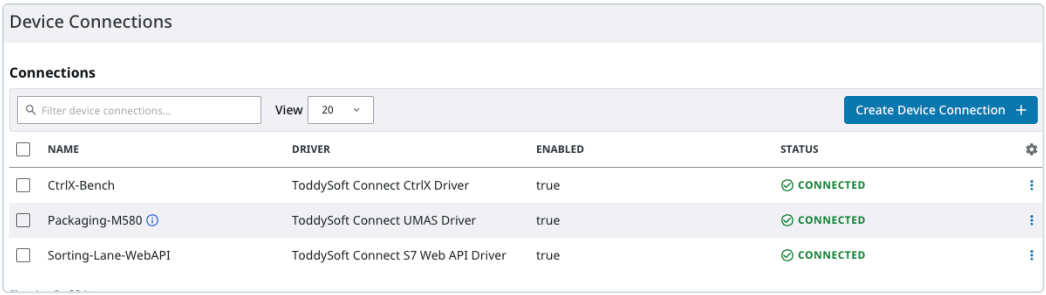


2. Click **Create new Device...** and select **ToddySoft Connect UMAS Driver** as the device type.



3. Configure the connection settings (see the sections below) and click **Create Device Connection**.

4. The new device then appears in the Device Connections list.



Finding devices on the network

The configuration form carries a **Discover Devices** button. Schneider controllers announce nothing, so the gateway **connects to each address in turn** on port 502 and asks whatever answers to identify itself over UMAS. Only a controller that answers that request is listed — a plain Modbus device on port 502 is not, since it has no UMAS; the Modbus module finds those.

Because that means connecting to every address in a network, **nothing is probed until you say so.** The first sweep after opening the dialog probes no range and reports which networks it left alone, under **Addresses not probed**. Then:

1. Optionally type what to probe into **Addresses to probe**: a network (`192.168.24.0/24`) or a comma-separated list of addresses (`192.168.24.10,192.168.24.11`). Leave it empty for the gateway's own networks.
2. Press **Probe addresses**. A later **Rescan** probes the same addresses again; closing the dialog forgets the go-ahead.

Selecting a result fills in **Host** and **Port**. The result also shows the name the controller was given in its project, where it reports one.

- **A range larger than 1024 addresses is refused** before anything is sent, and named under **Range too large**; give a smaller one.
- **The timeout covers the whole probe.** If it runs out, the list shows what answered in time and says so under **Ran out of time**; raise the timeout or narrow the range.
- **Probing crosses routed networks**, unlike a broadcast: a controller on another subnet or across a VPN is found as long as the gateway can reach its port 502.
- **Probing is visible to the network.** An intrusion-detection system may report it as a port scan; agree it with whoever runs the network before probing a production network.

> There is no *Test Connection* action for this driver: create the device and read its status.

Running in Docker: the discovery agent

A gateway in a Docker container sees only the container's networks, not the plant's: an empty **Addresses to probe** means the container's own network, where there is nothing to find. A range you type works without the agent, because probing crosses routed networks; with the agent, an empty field probes the plant machine's own networks instead. The **ToddySoft discovery agent** closes that gap. It is a small, separate container that runs on the same machine with the host's network and searches the plant networks on the gateway's behalf. The agent only searches: connecting, browsing and reading stay with the gateway, through the machine's normal routing. A gateway installed directly on the plant machine, or a container on the host network, does not need it.

Installing it. The agent comes as an image archive next to this module, in the share you download the module from: `discovery-agent-<version>-<architecture>.tar.gz`, for `amd64` or `arm64`. Use the agent of the module's version. On the machine that runs the Ignition container (Linux):

```
docker load < discovery-agent-1.3.0-amd64.tar.gz
docker run -d --name toddysoft-discovery-agent \
  --restart unless-stopped --network host --cap-add NET_RAW \
  toddysoft/discovery-agent:1.3.0
```

`--network host` lets it search the plant networks, and `NET_RAW` lets it send and capture the discovery frames. Docker Desktop (macOS, Windows) runs containers in a virtual machine, whose networks the agent would search instead of the plant's.

Nothing to set in Ignition. On every **Discover Devices** the module looks for the agent at its container's default gateway, which on Docker is the machine itself. An agent started later is found on the next search, with no restart. A line above the results says what happened:

- *via discovery agent*: the agent searched; its devices are listed together with any the gateway found itself, each once.
- *no discovery agent found · container networks only*: no agent runs; the search is the gateway's own, as before.
- *discovery agent refused: …* or *discovery agent not usable: …*: the reason follows, for example the licence or the shared secret below.

Nothing is probed without your go-ahead, through the agent too: it probes only after **Probe addresses**, and only the addresses typed there or, with the field empty, its machine's own networks.

Licence. The agent searches only for a gateway licensed for this driver. With each search the module sends the licence it runs on, the bundled trial included, over an encrypted connection; the agent checks it, uses it for that search and keeps nothing.

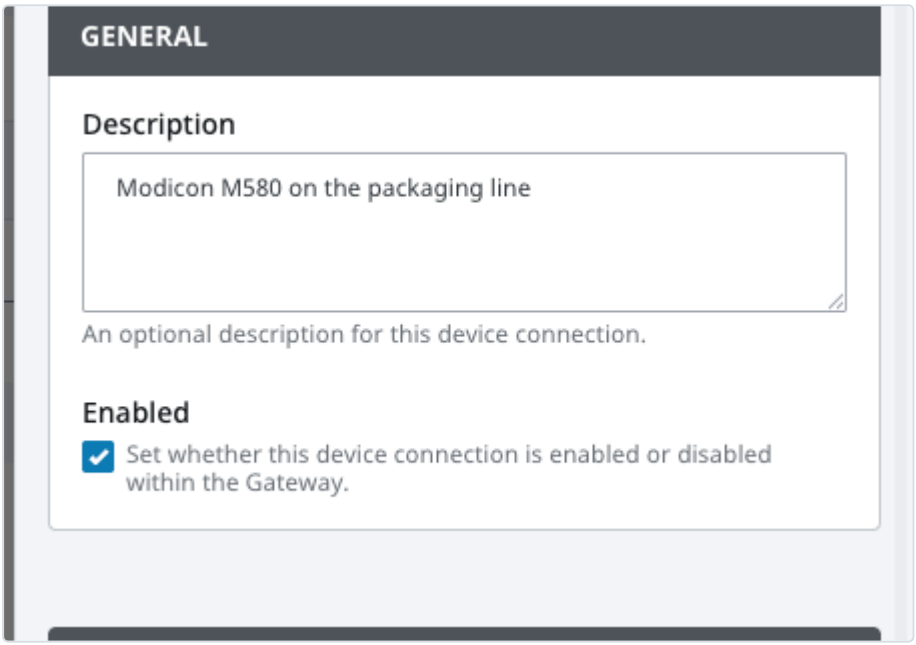
Security. The agent listens only on the Docker networks' own addresses on that machine (port 47900), never on a plant network, and accepts connections only from the container networks. To restrict it further, give it a shared secret: `-e TODDYSOFT_AGENT_SECRET=<secret>` on the agent and `-e TODDYSOFT_DISCOVERY_AGENT_SECRET=<secret>` on the Ignition container. With a secret set, a gateway without it, or with another, is refused. To stop the module looking for an agent, set `TODDYSOFT_DISCOVERY_AGENT=off` on the Ignition container.

Setting blocks

The configuration is organized into categories: **General**, **Transport**, **UMAS**, **Limits** and **Audit Log**.

General

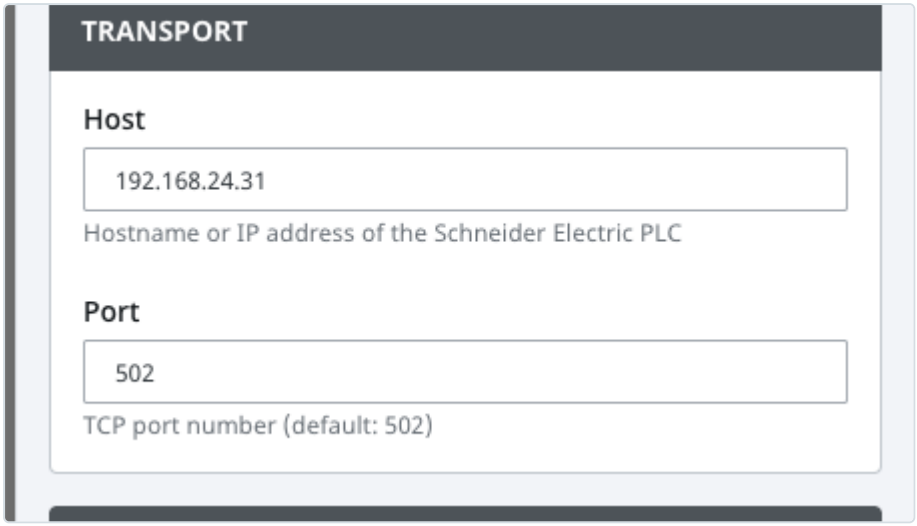
Assigns a name (and optional description) to the connection and controls whether it is enabled.



Setting	Description	Default
Name	Unique name for this device connection	
Description	Optional description for this connection	
Enabled	Whether the connection is active	`true`

This is the name the connection is then referenced under in other parts of Ignition.

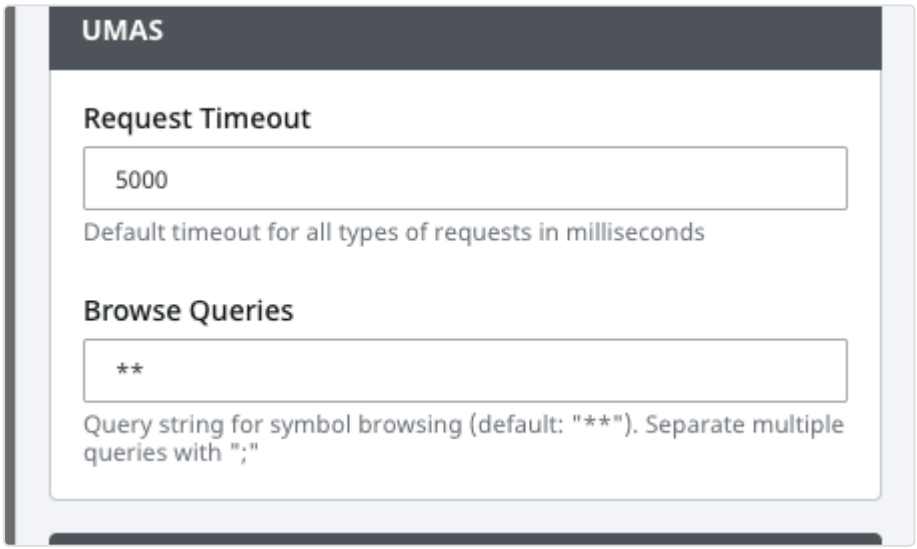
Transport



Setting	Description	Default
Host	Hostname or IP address of the Schneider Electric PLC	
Port	TCP port number	`502`

There is no transport selector, and none is missing: UMAS runs over plain TCP, and there is no TLS variant to choose. Port 502 is the Modbus port, shared as described above.

UMAS



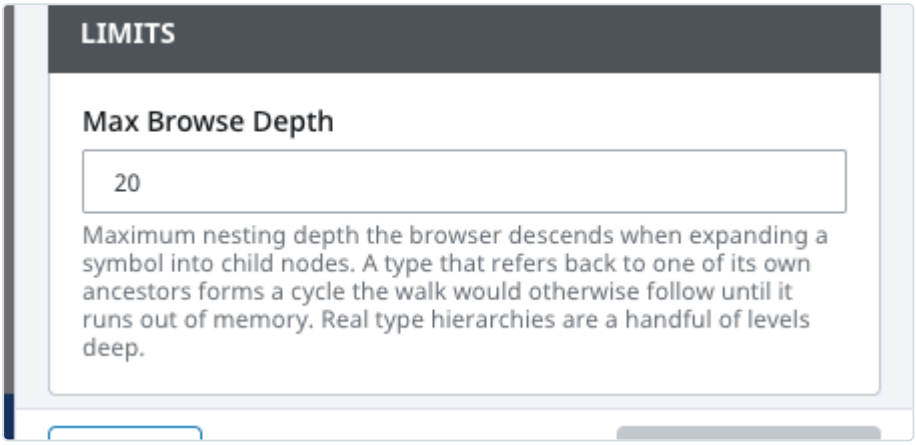
Setting	Description	Default
Request Timeout	Milliseconds allowed for one request	`5000`

Setting	Description	Default
Browse Queries	Query string for symbol browsing; separate multiple queries with `;`	`**`

Browse Queries

The default `**` returns the whole data dictionary. On a controller with a large program that is a lot of nodes; narrow it to the symbols you actually want rather than browsing everything and filtering in the Designer.

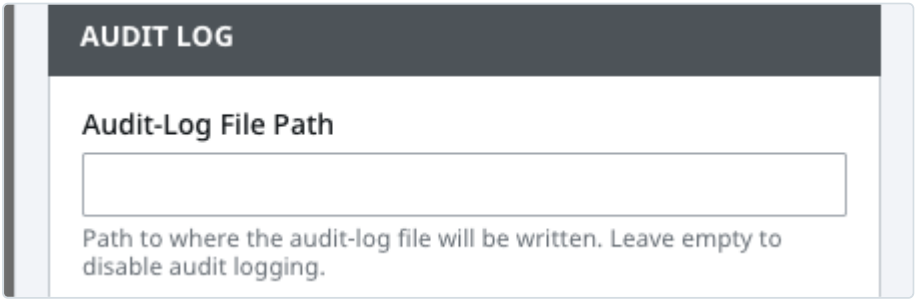
Limits



Setting	Description	Default
Max Browse Depth	How far the browser descends when expanding a symbol into children	`20`

This is a **cycle guard, not a performance knob**. A user-defined type that refers back to one of its own ancestors forms a loop the browse walk would otherwise follow until it runs out of memory. Real type hierarchies are a handful of levels deep, so the default of `20` sits far above anything legitimate and only ever stops a structure that could not terminate.

Audit Log



Setting	Description	Default
Audit-Log File Path	Path to a file protocol audit events are written to; empty disables	

The audit log records protocol-level events and grows quickly, so enable it only while investigating a problem.

Tag Addressing

UMAS addresses are **symbolic** — the names from the PLC program — with dotted paths for struct fields and subscripts for array elements:

```
<symbol>(.<field>)*([<index>(,<index>)*])*
```

Address	Meaning
`g_s16`	A standalone `INT` variable
`g_string1`	A standalone `STRING`
`MyStruct.RealField`	A struct field
`MyArray[5]`	Element 5 of a one-dimensional array
`MyMatrix[2][3]`	Cell `[2][3]` of a two-dimensional array
`MyStruct.Substruct.Field[10]`	A nested struct path with array indexing

Names are case-sensitive and must match what Control Expert exported. Browse to discover them rather than typing them from the program listing — the browse returns the controller's actual data dictionary, which is the authority.

`STRING` buffer sizes are derived from the symbol table, not assumed, so a string reads at its real declared length.

After a Control Expert download

The browse result is cached and refreshed when the controller's program CRC changes. Immediately after downloading a changed program the cache can be one poll cycle stale; wait a cycle and browse again rather than concluding a symbol is missing.

Tag Groups

By default, all tags are collected in the default tag group, which means all selected tags are fetched at the default interval of once a second.

If different intervals are desired, create additional tag groups, configure the desired intervals there, and assign tags to the corresponding tag groups.

UMAS has no change notification on the wire, so subscriptions are served by polling. The tag group interval *is* the poll rate, and a fast group on many tags is real load on the controller.

Licence diagnosis

When a device reports a licence problem, the module explains it on the gateway itself. Open the device's settings (**Config > OPC UA > Device Connections**, edit the device, or create a new one): the **Licence** card at the top of the form shows the diagnosis. It is on the form for a new device too, so the licence can be checked before the first device exists.

The diagnosis shows:

- **which gateway** this is, and its role if it is part of a redundant pair;
- **Ignition's licence** for this module on this gateway: activated, trial running (with its end) or trial expired;
- **what ToddySoft licence data arrived** with Ignition's licence: how many parts, whether they are complete, and a checksum that identifies the licence without revealing it;
- the **driver's own verdict** and the **cause**, in words.

Cause	Meaning	What to do
`IGNITION_UNLICENSED`	Ignition applies no activated licence to this module on this gateway (trial running or expired). ToddySoft licence data only arrives with an activated licence.	Activate a licence key that covers this module on this gateway. In a redundant pair every node needs its own activation. If a key was activated and this cause remains, see Troubleshooting: an older, expired key covering this module may be the one Ignition judges it by.
`NO_DATA_FOR_DRIVER`	The activated key carries no ToddySoft licence for this driver.	Send the report to ToddySoft.
`DATA_INCOMPLETE`	The licence data arrived incomplete or damaged.	Send the report to ToddySoft; the licence has to be delivered again.

Cause	Meaning	What to do
`DATA_REJECTED`	The driver refused the delivered licence; the reason is shown.	Send the report to ToddySoft.
`RUNNING_ON_BUNDLED_TRIAL`	A licence was delivered but not accepted, and the driver runs on its built-in trial until that ends.	Send the report to ToddySoft.
`NOT_CHECKED`	Licence data arrived, but the driver gave no verdict - for instance because it could not be loaded.	Send the report to ToddySoft.
`VALID`	Licensed; tier and expiry are shown.	Nothing.
`NOT_YET_KNOWN`	The first seconds after start-up, before Ignition reports a licence state.	Wait a moment.

Sending a report to support. The diagnosis comes as a plain-text report, the one thing to send with any licence question. **Copy report** puts it on the clipboard; **Show report** displays it. The same text is served at `http://<gateway>:<port>/data/com.toddysoft.connect.umas/licence-report` to a logged-in user. It contains no licence data, no keys, no credentials and no customer details. In a redundant pair, send the report from both nodes.

The gateway log carries the same report once each time the licence situation changes.

Troubleshooting

Licensing problems after a new or renewed licence key was activated — `DATA_INCOMPLETE` with "duplicate fragment index", or `IGNITION_UNLICENSED` with "trial expired", in the licence diagnosis:

- **Two licence keys covering this module are activated on the gateway at the same time**, typically a renewed or upgraded key activated alongside the old one.

Ignition hands the module the licence data of both, in numbered parts that carry the same numbers; the module cannot tell which parts belong together, keeps one of each pair, and the licence it reassembles is unreadable. Depending on the parts it keeps, devices are refused or run on the module's bundled trial until that ends.
- If one of the two keys has **expired**, Ignition may judge the module by that key alone: the diagnosis shows `IGNITION_UNLICENSED` ("trial expired - no activated licence applies to this module") and "Data: none received", although the new key was activated successfully.
- Keep exactly **one** activated licence key that covers this module per gateway: deactivate the old one under **Platform > Licensing**, then restart the module (or the gateway). In a redundant pair, check every node.

The module will not load, or the log shows `ClassNotFoundException` / `NoClassDefFoundError`:

- Ignition's **OPC UA** module is missing or disabled. This driver is an OPC UA device driver and cannot load without it. Install or re-enable it under **Config > System > Modules** and restart the gateway — see Installation

Device shows "Faulted":

- Verify the PLC is reachable and that TCP port 502 is open through any site firewall
- Confirm the PLC program is running — without it there is no data dictionary
- Check whether a site security policy has disabled UMAS on the controller; the TCP connection then succeeds while the protocol exchange does not
- Review Gateway logs: **Status > Diagnostics > Logs**

`INVALID_ADDRESS` from a read or write:

- The symbol does not exist in the *current* program, or the dotted path / array index has a typo. Browse to confirm the actual name, and remember that names are case-sensitive

Nothing to browse, although the device connects:

- The project was built without the data dictionary, so the controller has no symbol table to hand out. Enable it in Control Expert and **rebuild and download** the project — see Before you start: the controller side
- The program is not running, or the data dictionary has not finished downloading

A symbol is missing right after a Control Expert download:

- The cached browse is refreshed on a program-CRC poll. Wait one cycle

Tags show bad quality:

- Verify the symbol still exists in the program after the latest download
- Check spelling and case

Subscriptions miss fast changes:

- Expected: there is no change notification in UMAS, so subscriptions are polled. Shorten the tag group interval, accepting the extra load

Docker related issues

The UMAS protocol carries no identity for this end of the connection, so a container's internal address is never announced to the controller and nothing has to be reconciled. A gateway in a bridged container connects normally, provided port 502 is routable.

Device discovery needs the discovery agent. *Discover Devices* searches the networks the gateway itself is attached to, which for a container are Docker's, not the plant's. Run the ToddySoft discovery agent next to the container — see Running in Docker: the discovery agent — or enter the host by hand.

VPN related issues

A remote controller reached over a VPN needs nothing special from this driver, for the same reason: the connection is an ordinary outbound TCP session. Only reachability matters.

Version history

Unreleased — 1.3.0

- First release of this manual.
- **New: Discover Devices** finds Schneider controllers by probing addresses, with your go-ahead. See Finding devices on the network.
- **New:** licence diagnosis — why a device is or is not licensed, shown on the gateway, with a report to send to support. See Licence diagnosis.
- **New:** troubleshooting for two licence keys covering this module activated on one gateway at the same time. See Troubleshooting.
- **New:** discovery when Ignition runs in Docker, through the ToddySoft discovery agent. See Running in Docker: the discovery agent.
- **Changed:** date and time tags have new value types. DATE and DATE_AND_TIME are DateTime values (until now a DateTime tag holding epoch milliseconds as a number); TIME and LTIME are milliseconds, TIME_OF_DAY and LTIME_OF_DAY milliseconds since midnight (Int64). Writes take the same forms. Adjust scripts and bindings that used the old values.
- **Fixed:** a write the PLC refuses now fails in Ignition, with the reason as the status (for example `Bad_TypeMismatch` for a value of the wrong type). Until now every write was reported as successful, and the refusal appeared only in the gateway log.

- **Fixed:** a tag that has not been read yet, or whose last read failed, no longer reads 0 with Good quality. Before its first read it has no value and the quality Uncertain; after a failed read it keeps its last value with Bad quality until the next good read.
- **Fixed:** removing the last tag of a tag group (or polling rate) no longer leaves the device showing *Connection Lost*. A poll still running at that moment made the device fail with `Batch '...' not found` in the gateway log and stay in that state until it was restarted.
- **Changed:** a browsed array is one folder, named after the array and its index range (for example `arr[0..9]`), holding its elements; until now the elements sat directly beside the array's neighbours. A multi-dimensional array, until now left out of the browse, appears as nested folders down to its elements (`cube[1..2][1..2]`, then `cube[1][1..2]`, then `cube[1][1]`). An array of structures shows a folder per element holding its members (for example `pts[2].x`). Each element keeps its OPC item path, so existing tags keep working.